

מגמת הנדסת תוכנה 5 יח"ל

Driver Moodle

מערכת לניהול שיעורי נהיגה

Web project databases and programming asynchronous services חלופה

שם התלמיד: אלי רפפורט

שם בית הספר: מקיף ז אשדוד

שם המנחה: אולגה גנדלמן

תאריך הגשה: 05/05/2026

במסגרת הפרויקט פיתחתי מערכת לניהול שיעורי נהיגה בשם Driver Moodle. המערכת נועדה לעזור לתלמיד, למורה ולמנהל לעבוד במקום אחד מסודר במקום להשתמש בהרבה הודעות, פתקים או קבצי אקסל.

המערכת כוללת שרת WCF מרכזי, בסיס נתונים Access, ושלושה לקוחות שונים: לקוח WPF למחשב, לקוח Web ב-Razor Pages, ולקוח MAUI שמתאים למובייל. כל הלקוחות עובדים מול אותו שירות, ולכן המידע נשמר באותו מקום ולא מתפזר בין כמה מערכות.

בפרויקט יש פעולות מרכזיות שמתאימות לעולם של לימודי נהיגה: הרשמה והתחברות, קביעת שיעור, ביטול שיעור, תשלום על שיעורים, צפייה בהתראות, שליחת הודעות, ניהול קורסים ופתיחת פניות תמיכה. לכל סוג משתמש יש מסכים שמתאימים לו, כדי שתלמיד, מורה ומנהל לא יעבדו באותו אופן.

בצד הנתונים השתמשתי בטבלאות כמו Student, Teacher, Lessons, Payments, Notifications ו-Tickets. דרך הטבלאות האלה המערכת שומרת תלמידים, מורים, שיעורים, תשלומים, הודעות ופניות. הגישה לבסיס הנתונים נעשית דרך שכבת ViewDB, כדי שהלקוחות לא יפנו ישירות ל-Access.

בצד האבטחה והבדיקות שמתי דגש על שמירת סיסמאות בצורה בטוחה יותר בעזרת PBKDF2, בדיקות קלט בסיסיות כמו שדות חובה ואימייל תקין, ושימוש בשאילתות עם פרמטרים כדי לא להכניס ערכים ישירות לתוך SQL. המטרה של הפרויקט הייתה לא רק לבנות מסכים, אלא לבנות מערכת שעובדת בצורה מלאה: משתמש נכנס, עושה פעולה במסך, השרת מקבל את הבקשה, הנתונים נשמרים בבסיס הנתונים, והמשתמש מקבל תשובה ברורה בחזרה. כך אפשר לראות את הקשר בין הלקוח, השרת ובסיס הנתונים.

תוכן עניינים

פרק	עמוד
חלק 1: מבוא	5
חלק 2: ניתוח המערכת	7
חלק 3: מפות ניווט	9
חלק 4: בסיס הנתונים	11
חלק 5: מבנה הפרויקט והמימוש	13
חלק 6: מדריך למשתמש	14
חלק 7: בדיקות ותיקונים	15
חלק 8: פירוט תהליכי עבודה	16
חלק 9: פירוט טכני נוסף	21
חלק 10: בדיקות מורחבות וקבצים מרכזיים	24
חלק 11: סיכום ונספחים	26

בדיקת דרישות המחווון

כדי שיהיה קל לבדוק את העבודה, ריכזתי כאן בקצרה איפה כל דרישה מופיעה בפרויקט.

#	דרישה	מענה בפרויקט
1	בסיס נתונים יחסי	Access עם טבלאות כמו Student, Teacher, Lessons, Payments ו-Notifications.
2	מבנה שכבות	שירות WCF, מודלים ושכבת ViewDB.
3	שירות רשת	IService1 ו-Service1 עם BasicHttpBinding.
4	פעולות CRUD	הוספה, שליפה, עדכון ומחיקה דרך מחלקות ViewDB.
5	קשרים ו-JOIN	קשרים בין תלמיד, מורה, שיעור ותשלום.
6	ממשק משתמש	שלושה לקוחות: WPF, Web ו-MAUI.
7	סוגי משתמשים	תלמיד, מורה ומנהל עם מסכים שונים.
8	בדיקות קלט	שדות חובה, אימייל, תאריכים וסיסמאות.
9	הרחבות	ירושה במודלים ובשכבת בסיס הנתונים, ואבטחת סיסמאות.
10	נושאים נוספים	Async, שאילתות פרמטריות, דוחות, התראות ו-ValueConverter.

חלק 1: מבוא

Driver Moodle היא מערכת לניהול שיעורי נהיגה. בחרתי לעשות את הפרויקט הזה כי בזמן שלמדתי נהיגה הרגשתי שקביעת שיעורים לא תמיד מסודרת. לפעמים הכול היה בהודעות, לפעמים שעה השתנתה, ולא תמיד היה ברור מה נשאר לשלם.

המטרה שלי הייתה לבנות מערכת אחת שבה תלמיד, מורה ומנהל יכולים לעבוד מול אותו מידע. התלמיד רואה שיעורים ותשלומים, המורה מנהל תלמידים ושיעורים, והמנהל יכול לראות נתונים כלליים ולעזור כשיש בעיה. המערכת בנויה משלושה לקוחות: WPF למחשב, Web ב-Razor Pages, ו-MAUI למובייל. כולם עובדים מול אותו שירות WCF ואותו בסיס נתונים Access.

מה הבעיה שהפרויקט פותר

- קביעת שיעורים דרך הודעות יכולה להיות לא מסודרת.
- קשה לעקוב אחרי שיעורים ששולמו ושיעורים שעדיין פתוחים.
- המורה והתלמיד לא תמיד רואים את אותה תמונה.
- אין מקום אחד שמרכז שיעורים, התראות, קורסים ופניות תמיכה.

קהל יעד

המערכת מיועדת לתלמידי נהיגה, מורי נהיגה ומנהלי בית ספר לנהיגה.

למה בחרתי דווקא בנושא הזה

בחרתי בנושא הזה כי הוא קרוב אליי. כשאני למדתי נהיגה, הרבה דברים היו תלויים בזיכרון ובהודעות. אם רציתי לבדוק מתי יש לי שיעור, הייתי צריך לחפש בשיחות. אם רציתי לדעת כמה שילמתי, זה לא תמיד היה במקום אחד. בגלל זה ניסיתי לבנות מערכת שלא מסבכת את המשתמש. הרעיון הוא שכל פעולה חשובה תהיה במקום ברור: שיעורים במסך שיעורים, תשלומים במסך תשלומים, הודעות במסך התראות ופניות במסך תמיכה.

מה רציתי להשיג

- מערכת שתהיה נוחה לתלמיד ולא רק למנהל.
- מקום אחד שבו אפשר לראות שיעורים ותשלומים.
- חלוקה ברורה בין תלמיד, מורה ומנהל.
- קוד שמחולק לשכבות כדי שיהיה אפשר להבין ולתחזק אותו.

מטרות טכנולוגיות

לחבר שלושה לקוחות שונים לאותו שירות WCF, כך שכל לקוח יוכל לעבוד מול אותם נתונים. לבנות שכבת נתונים מסודרת ב-Access, עם טבלאות נפרדות לשיעורים, תשלומים, תלמידים, מורים והתראות. להשתמש בשכבת ViewDB כדי להפריד בין הקוד של המסכים לבין הקוד שפונה לבסיס הנתונים. לשלב בדיקות קלט בסיסיות כדי להפחית שגיאות בזמן שימוש רגיל במערכת.

מטרות פונקציונליות

לאפשר לתלמיד לראות שיעורים, לשלם ולקבל התראות במקום אחד.

לאפשר למורה לנהל תלמידים, שיעורים, קורסים והודעות בצורה מסודרת.

לאפשר למנהל לעקוב אחרי פעילות כללית ולטפל בפניות תמיכה.

לשמור היסטוריה של פעולות חשובות, כדי שיהיה אפשר לחזור ולבדוק מה קרה במערכת.

חלק 2: ניתוח המערכת

במערכת יש שלושה סוגי משתמשים. כל אחד מהם מקבל מסכים שמתאימים לתפקיד שלו, כדי שלא תהיה גישה למסכים שלא קשורים אליו.

משתמש	מה הוא עושה במערכת
תלמיד	קובע שיעורים, רואה שיעורים קיימים, משלם, מקבל התראות ופונה לתמיכה.
מורה	מנהל תלמידים, שיעורים, הודעות, קורסים ודוחות.
מנהל	בודק נתונים כלליים, מטפל בפניות ובודק פעילות במערכת.

מצב קיים מול מצב אחרי המערכת

מצב קיים	מצב אחרי הפרויקט
הודעות מפוזרות בוואטסאפ.	לוח שיעורים מסודר בתוך המערכת.
מעקב ידני אחרי תשלומים.	תשלומים ושיעורים נשמרים בבסיס הנתונים.
קשה לדעת מה השתנה.	התראות נשלחות למשתמש המתאים.
אין מקום מסודר לפניות.	יש פניות תמיכה עם סטטוס וטיפול.

תרחישי שימוש מרכזיים

- הרשמה והתחברות: המשתמש מזין פרטים, המערכת בודקת תקינות ומזהה את התפקיד שלו.
- קביעת שיעור: תלמיד בוחר תאריך ושעה, והשיעור נשמר עם המורה שלו.
- ביטול שיעור: שיעור מסומן כמבוטל ונשלחת התראה למורה.
- תשלום: תלמיד משלם על שיעור, והמערכת מעדכנת את סטטוס התשלום.
- פנייה לתמיכה: משתמש פותח פנייה והמנהל יכול לענות ולסגור אותה.

זרימת מידע כללית

הלקוח שולח בקשה לשירות WCF. השירות בודק את הנתונים, פונה לשכבת ViewDB, ושם מתבצעת העבודה מול Access דרך OleDb. התשובה חוזרת ללקוח ומוצגת במסך המתאים.

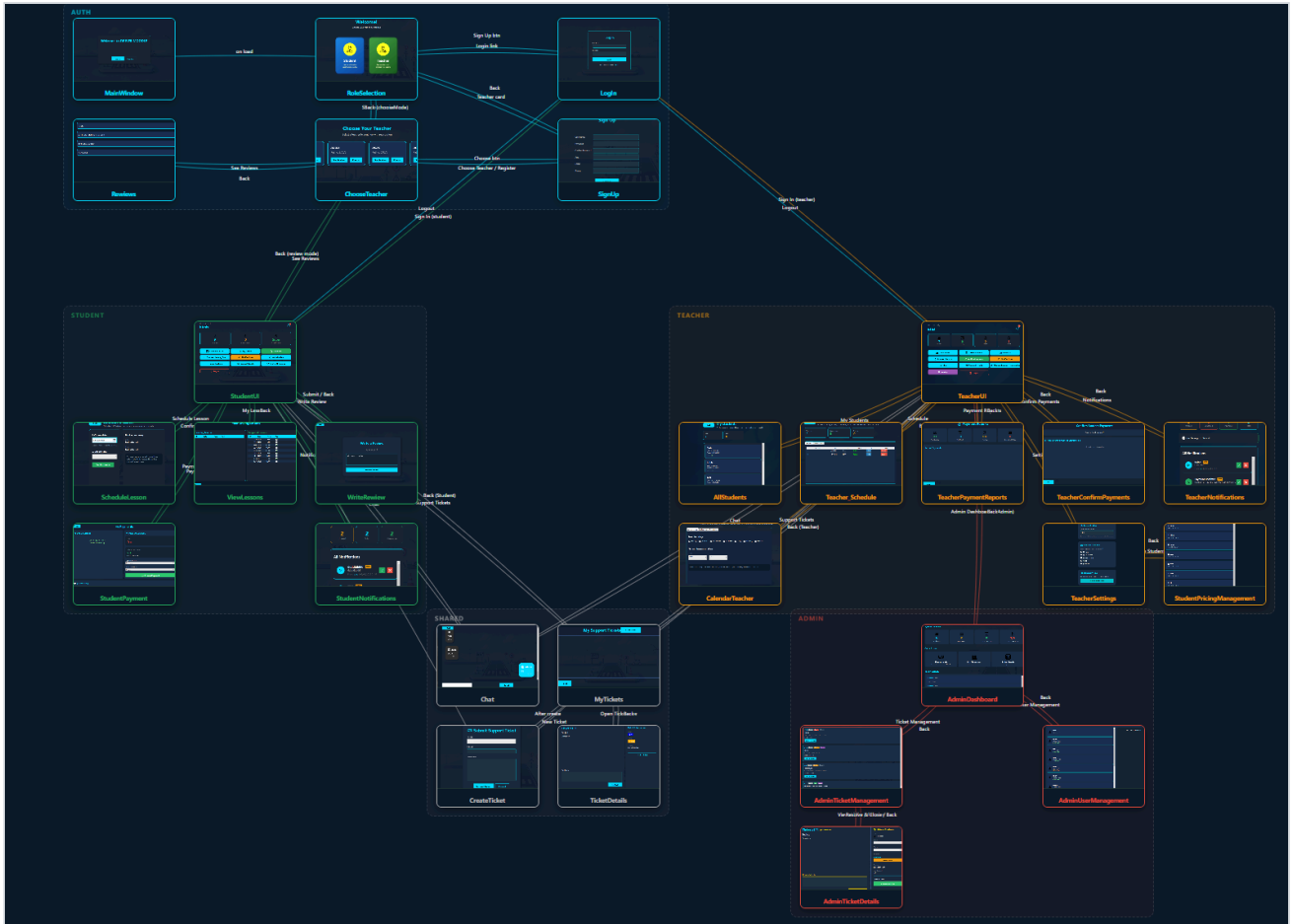
שלב	מה קורה בפועל
1	המשתמש לוחץ על פעולה במסך, למשל קביעת שיעור.
2	הלקוח בודק שהשדות הבסיסיים לא ריקים.
3	הלקוח שולח בקשה ל-Service1 דרך WCF.
4	השרת מבצע בדיקות נוספות ופונה למחלקת ViewDB המתאימה.
5	הנתונים נשמרים או נשלפים מ-Access.
6	התוצאה חוזרת למסך והמשתמש רואה הודעה או רשימה מעודכנת.

דרישות עיקריות מהמערכת

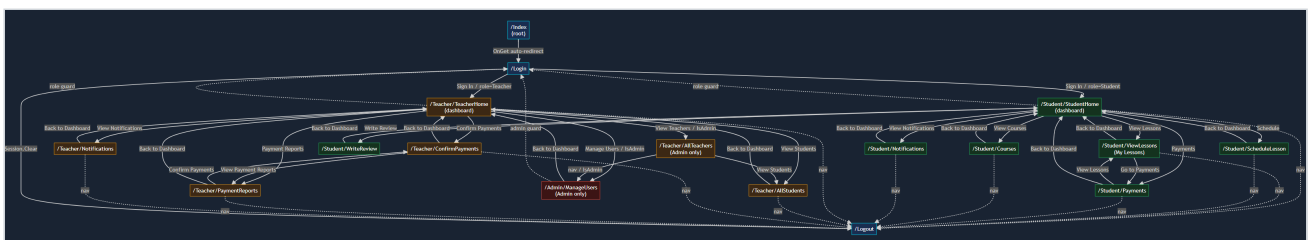
- המשתמש צריך להיכנס למערכת לפי שם משתמש וסיסמה.
- תלמיד צריך לראות רק את הנתונים שלו.
- מורה צריך לראות רק את התלמידים והשיעורים שקשורים אליו.
- מנהל צריך לקבל מסכים רחבים יותר לניהול ובקרה.
- כל פעולה חשובה צריכה להישמר בבסיס הנתונים ולא רק במסך.
- כאשר יש פעולה שמשפיעה על משתמש אחר, למשל ביטול שיעור, צריך ליצור התראה.

חלק 3: מפות ניווט

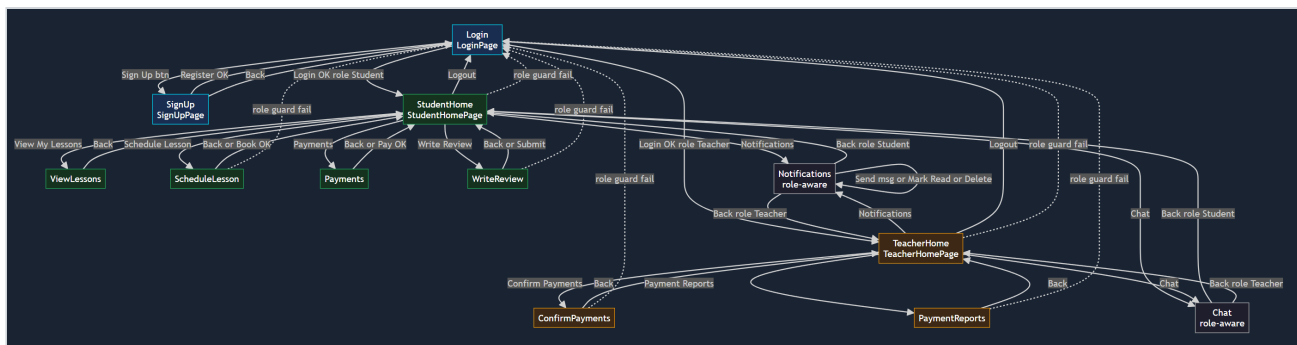
הכנתי מפות ניווט כדי שיהיה ברור איך עוברים בין המסכים בכל לקוח. לא כל לקוח נראה אותו דבר, אבל כולם עובדים מול אותו שרת.



מפת ניווט כללית של לקוח WPF



מפת ניווט כללית של לקוח Web



מפת ניווט כללית של לקוח MAUI

ב-WPF רוב העבודה מתבצעת דרך חלונות ומסכים. ב-Web העבודה מחולקת לדפים לפי תפקיד. ב-MAUI השתמשתי בניווט שמתאים יותר למובייל.

הסבר על הניווט ב-WPF

ב-WPF המשתמש מתחיל במסך התחברות או בחירת תפקיד. אחרי התחברות, המערכת מעבירה אותו למסך שמתאים לו. תלמיד מגיע למסכי תלמיד, מורה מגיע למסכי מורה, ומנהל מקבל אפשרויות ניהול.

היה לי חשוב שהמשתמש לא יצטרך לנחש לאן ללכת. לכן המסכים מחולקים לפי פעולות: שיעורים, תשלומים, התראות, קורסים ותמיכה.

הסבר על הניווט ב-Web

ב-Web השתמשתי בדפים נפרדים. זה מתאים יותר למסך ניהול, כי אפשר לפתוח דף לכל פעולה. גם כאן יש בדיקה לפי תפקיד, כדי שתלמיד לא ייכנס בטעות לדף שמיועד למורה או למנהל.

הסבר על הניווט ב-MAUI

ב-MAUI המטרה הייתה לתת גרסה נוחה יותר למובייל. לא כל מסך חייב להיות זהה ל-WPF, אבל הפעולות המרכזיות נשארות דומות: כניסה, צפייה בשיעורים, תשלום והתראות.

חלק 4: בסיס הנתונים

בסיס הנתונים נשמר בקובץ Access מסוג accdb. השתמשתי בכמה טבלאות כדי לא לשמור הכול במקום אחד. כך קל יותר לעדכן שיעור, תשלום, תלמיד או התראה בלי לפגוע בשאר הנתונים.

טבלה	תפקיד
Student	שמירת פרטי תלמידים וקישור למורה.
Teacher	שמירת פרטי מורים ונתוני עבודה.
Lessons	שמירת שיעורי נהיגה, תאריך, שעה, תלמיד ומורה.
Payments	שמירת תשלומים על שיעורים.
Notifications	שמירת הודעות והתראות למשתמשים.
Tickets	שמירת פניות תמיכה ותגובות.
Courses / Modules	שמירת קורסים, מודולים והתקדמות תלמיד.

קשרים בין טבלאות

- כל תלמיד מקושר למורה.
- כל שיעור מקושר לתלמיד ולמורה.
- כל תשלום מקושר לשיעור ולתלמיד.
- התראה נשמרת לפי משתמש מקבל וסוג משתמש.

גישה לבסיס הנתונים

הלקוח עצמו. SELECT, INSERT, UPDATE ו-DELETE אחראיות על ViewDB המחלקות בשכבת OleDb. נעשית דרך Access-הגישה ל
לא פונה ישירות לבסיס הנתונים.

Provider=Microsoft.ACE.OLEDB.12.0;Data Source=UsersDataBase.accdb

דוגמאות לשדות חשובים

טבלה	שדות לדוגמה	למה הם משמשים
Student	id, username, password, teacherId	זיהוי תלמיד, התחברות וקישור למורה.
Teacher	id, username, price, rating	שמירת פרטי מורה, מחיר ודירוג.
Lessons	LessonID, StudentID, TeacherID, Date, Hour, Paid	שמירת שיעור והאם הוא שולם.
Payments	PaymentId, StudentID, LessonId, Amount	שמירת תשלום לפי תלמיד ושיעור.
Notifications	Id, RecipientId, Message, IsRead	שמירת התראות שנשלחות למשתמש.

למה לא שמרתי הכול בטבלה אחת

אם כל הנתונים היו בטבלה אחת, היה קשה לעדכן ולבדוק אותם. למשל, שיעור צריך לדעת מי התלמיד ומי המורה, אבל פרטי התלמיד עצמם לא צריכים להישמר שוב בכל שיעור. לכן חילקתי את הנתונים לטבלאות שונות וקישרתי ביניהן.

פעולות בסיסיות על הנתונים

- הוספה: יצירת תלמיד, שיעור, תשלום או התראה.
- שליפה: הצגת שיעורים, תלמידים, תשלומים ודוחות.
- עדכון: שינוי סטטוס שיעור, סימון תשלום או סימון התראה כנקראה.
- מחיקה או ביטול: מחיקה במקומות שמתאימים, ובמקרים אחרים סימון כפעולה שבוטלה.

חלק 5: מבנה הפרויקט והמימוש

הפרויקט מחולק לשכבות כדי שיהיה ברור איפה נמצא כל חלק. הלקוחות אחראים על המסכים. השרת אחראי על הפעולות. שכבת ViewDB אחראית על בסיס הנתונים.

שכבה	מה נמצא בה
לקוח	WPF, Web ו-MAUI. כאן נמצאים המסכים והפעולות שהמשתמש רואה.
שירות WCF	IService1 ו-Service1. כאן נמצאות הפעולות שהלקוחות קוראים להן.
Model	מחלקות הנתונים כמו Student, Teacher, Lesson ו-Payment.
ViewDB	מחלקות שמבצעות פעולות מול Access.

דוגמה לפעולה במערכת

כאשר תלמיד קובע שיעור, הלקוח שולח בקשה לשרת. השרת מקבל את הנתונים, בודק אותם, ושומר את השיעור בטבלת Lessons. אחרי זה הלקוח מקבל תשובה ומציג את השיעור במסך.

פעולה	איפה היא מתחילה	איפה היא נשמרת
קביעת שיעור	מסך תלמיד ב-WPF / Web / MAUI	Lessons
תשלום	מסך תשלומים	Lessons וגם Payments
שליחת הודעה	מסך מורה או תמיכה	Notifications / Tickets
ניהול קורס	מסך מורה	Courses ו-CourseModules
דוח	מסך מורה או מנהל	שליפה וחישוב מהטבלאות

החזרה של השירות

IService1 הוא המקום שבו מוגדרות הפעולות שהלקוחות יכולים לקרוא להן. Service1 הוא המימוש של הפעולות האלה. כך הלקוח לא צריך לדעת איך בדיוק נשמרים הנתונים בבסיס הנתונים.

```
public interface IService1
{
    bool CheckUserPassword(string username, string password);
    bool AddLessonForStudent(Lesson lesson);
    List<Notification> GetUserNotifications(int userId, string userType);
}
```

מחלקות Model

מחלקות ה-Model מייצגות את הנתונים שעוברים בין הלקוח לשרת. לדוגמה Student מייצג תלמיד, Lesson מייצג שיעור, ו-Payment מייצג תשלום. השתמשתי במחלקות כאלה כדי לא להעביר נתונים בצורה לא מסודרת.

שאליות SQL

בפרויקט יש שאליות פשוטות וגם שאליות מורכבות יותר. לדוגמה: שליפת שיעורים לפי תלמיד, עדכון תשלום, חישוב הכנסות, ושליפה של התראות לפי משתמש.

```
SELECT * FROM Lessons WHERE StudentID = @StudentID
```

אבטחה ובדיקות קלט

לא שמרתי סיסמאות כטקסט רגיל. השתמשתי ב-PBKDF2 כדי לשמור סיסמה בצורה בטוחה יותר. בנוסף, בקלט מהמשתמש יש בדיקות כמו שדות חובה, אימייל תקין ותאריכים הגיוניים.

בשאלות השתמשתי בפרמטרים במקום לחבר טקסט ישירות לשאלה. זה עוזר למנוע הכנסת קוד SQL דרך שדות הקלט.

דוגמה לשאלה עם פרמטר

```
command.CommandText = "SELECT * FROM Student WHERE username=@username";  
command.Parameters.AddWithValue("@username", username);
```

בצורה הזאת הערך שהמשתמש מזין לא נכנס ישירות לתוך הטקסט של השאלה, אלא נשלח כפרמטר.

חלק 6: מדריך למשתמש

תלמיד

- נכנס למערכת עם שם משתמש וסיסמה.
 - רואה שיעורים שנקבעו לו.
 - יכול לקבוע שיעור חדש אם יש שעה פנויה.
 - יכול לשלם על שיעורים ולראות היסטוריה.
 - מקבל התראות מהמורה או מהמערכת.
- לדוגמה, תלמיד שנכנס למערכת יכול לבדוק מתי השיעור הבא שלו, אם הוא כבר שילם עליו, ואם יש הודעה חדשה מהמורה. זה חוסך חיפוש בהודעות ומרכז את המידע במסך אחד.

מורה

- רואה את התלמידים שלו.
 - מנהל שיעורים ותשלומים.
 - שולח הודעות והתראות לתלמידים.
 - יכול לנהל קורסים ומודולים.
- המורה משתמש במערכת כדי לעקוב אחרי תלמידים ושיעורים. במקום לזכור ידנית מי שילם ומי צריך שיעור נוסף, הנתונים נשמרים במערכת ואפשר לשלוף אותם לפי הצורך.

מנהל

- רואה נתונים כלליים על המערכת.
 - מטפל בפניות תמיכה.
 - בודק פעילות משתמשים ועוזר בבעיות.
- המנהל לא עובד כמו תלמיד או מורה. הוא צריך תמונה רחבה יותר של המערכת, בעיקר בשביל טיפול בבעיות ובקרה כללית.

פירוט קצר של מסכים במערכת

- מסך התחברות: מאפשר למשתמש להכניס שם משתמש וסיסמה. לאחר בדיקה, המערכת מעבירה אותו למסך המתאים לתפקיד שלו.
- מסך שיעורים: מציג שיעורים קיימים ומאפשר קביעת שיעור חדש או ביטול שיעור לפי ההרשאות.
- מסך תשלומים: מציג שיעורים שצריך לשלם עליהם ושומר את פעולת התשלום בבסיס הנתונים.
- מסך התראות: מציג הודעות שנשלחו למשתמש, למשל הודעה מהמורה או עדכון על שינוי בשיעור.
- מסך תמיכה: מאפשר לפתוח פנייה ולנהל שיחה על בעיה עד שהיא נסגרת.
- הודעות שגיאה ומשוב למשתמש
- כאשר המשתמש מזין פרטים לא נכונים, המערכת לא אמורה לקרוס. היא צריכה להציג הודעה פשוטה וברורה. לדוגמה: אם הסיסמה קצרה מדי או אם שדה חובה ריק, המשתמש מקבל הודעה ויכול לתקן.

חלק 7: בדיקות ותיקונים

בדקתי את המערכת לפי פעולות שהמשתמשים אמורים לבצע באמת. המטרה הייתה לראות שהזרימה עובדת ולא רק שהקוד מתקמפל.

בדיקה	מה נבדק	תוצאה
התחברות	שם משתמש וסיסמה נכונים ושגויים.	המערכת מכניסה משתמש נכון וחוסמת פרטים שגויים.
קביעת שיעור	בחירת תאריך, שעה ומורה.	השיעור נשמר ומופיע ברשימה.
ביטול שיעור	סימון שיעור כמבוטל.	השיעור מתעדכן ונשלחת התראה.
תשלום	תשלום על שיעור פתוח.	סטטוס התשלום מתעדכן.
פניית תמיכה	פתיחת פנייה ותגובה.	הפנייה נשמרת ומוצגת למנהל.

תיקונים שבוצעו

- שיפרתי ניווט בין מסכים כדי שמשתמש לא יגיע למסך שלא מתאים לתפקיד שלו.
- סידרתי תצוגות שהיו נראות לא טוב בגדלים שונים של חלון.
- הוספתי בדיקות קלט כדי להפחית שגיאות בזמן שימוש.
- בדקתי שהודעות והתראות נשמרות בבסיס הנתונים ונשלפות לפי המשתמש הנכון.

בדיקות קצה שעשיתי

- ניסיון התחברות עם סיסמה לא נכונה.
- ניסיון לשמור שיעור בלי לבחור תאריך.
- ניסיון לשלם על שיעור שכבר סומן כשולם.
- כניסה למסך שלא מתאים לתפקיד המשתמש.
- שליחת פנייה ריקה לתמיכה.

הבדיקות האלה חשובות כי הן בודקות שימוש אמיתי ולא רק מצב שבו המשתמש עושה הכול נכון.

מה עדיין אפשר לשפר

יש דברים שהייתי משפר אם היה יותר זמן: עיצוב מסכים אחיד יותר בכל הלקוחות, יותר דוחות למנהל, וחיבור ליומן חיצוני כדי שהתלמיד יוכל לקבל תזכורת גם מחוץ למערכת.

בחרתי לא להכניס את כל הקוד לתוך הספר, כי זה היה הופך את המסמך לפחות קריא. במקום זה הכנסתי הסברים, טבלאות, קטעים קצרים והפניות לקבצים המרכזיים.

בחרתי להשאיר את כל הלקוחות מול אותו שירות WCF כדי שלא יהיו כמה מקורות מידע שונים. בנוסף, העדפתי לשמור פעולות כמו ביטול שיעור ותשלום בבסיס הנתונים, כדי שאפשר יהיה לבדוק אותן גם אחרי שהמסך נסגר.

החלטות שעשיתי במהלך העבודה

קובץ או תיקייה	תפקיד
IService1.cs	מגדיר את הפעולות שהלקוחות יכולים לקרוא להן.
Service1.cs	מממש את הפעולות של השרת.
ViewDB	כולל מחלקות שעובדות מול Access.
Model	כולל מחלקות נתונים כמו Student, Teacher, Lesson ו-Payment.
driver-client	לקוח WPF למחשב.
Driver/Pages	דפי Web ב-Razor Pages.
driver-maui	לקוח MAUI למובייל.
nav-map	מפות ניווט ותרשימים של המסכים.

קבצים מרכזיים בפרויקט

בדקתי שהמערכת שומרת התראה למשתמש הנכון, ושמשמש רואה רק את ההתראות שלו. בנוסף בדקתי פתיחת פנייה לתמיכה, תגובה של מנהל וסגירת פנייה.

בדיקות התראות ותמיכה

בדיקה	מה נבדק	למה זה חשוב
קביעת שיעור	שמירת שיעור חדש בטבלת Lessons.	כדי לוודא שהשיעור לא נשאר רק במסך.
ביטול שיעור	סימון שיעור כמבוטל.	כדי לשמור היסטוריה ולא למחוק מידע חשוב.
תשלום	שמירת תשלום ועדכון שיעור.	כדי לדעת אילו שיעורים שולמו.
דוח תשלומים	שליפת נתונים מטבלאות תשלומים ושיעורים.	כדי לבדוק שהחישובים מסתמים על נתונים אמיתיים.

בדיקות שיעורים ותשלומים

בדיקה	מה ניסיתי	תוצאה צפויה
שם משתמש תקין וסיסמה תקינה	כניסה של תלמיד או מורה.	המשתמש נכנס למסך המתאים.
סיסמה לא נכונה	הכנסת סיסמה שגויה.	המערכת לא מכניסה את המשתמש ומציגה שגיאה.
שדות ריקים	ניסיון כניסה בלי למלא פרטים.	המערכת מבקשת להשלים פרטים.

בדיקות התחברות

בחלק הזה אני מפרט עוד בדיקות וקבצים מרכזיים בפרויקט. זה עוזר להראות שהמערכת נבדקה לפי פעולות אמיתיות, ולא רק לפי כתיבת קוד.

חלק 10: בדיקות מורחבות וקבצים מרכזיים

למדתי שכאשר מחלקים את הפרויקט לשכבות, הרבה יותר קל להבין איפה לתקן בעיה. אם יש בעיה במסך, מחפשים בלקוח. אם יש בעיה בפעולה, בודקים את השירות. אם יש בעיה בנתונים, בודקים את ViewDB ואת Access.

מה למדתי מהמבנה הזה

בפרויקט יש בדיקות קלט כדי לא לאפשר שמירה של נתונים לא תקינים. בנוסף, הסיסמאות נשמרות בצורה בטוחה יותר בעזרת PBKDF2, והשימוש בפרמטרים בשאליות עוזר להגן מפני הכנסת SQL דרך שדות קלט.

בדיקות קלט ואבטחה

שכבת ViewDB אחראית על העבודה מול Access. כל מחלקה מטפלת בנושא אחר: תלמידים, מורים, שיעורים, תשלומים, התראות או פניות. זה עוזר לשמור על סדר בקוד.

שכבת ViewDB

פעולה	תפקיד
CheckUserPassword	בדיקת התחברות של משתמש.
AddLessonForStudent	הוספת שיעור לתלמיד.
Pay	שמירת תשלום ועדכון שיעור.
CancelLesson	סימון שיעור כמבוטל.
GetUserNotifications	שליפת התראות למשתמש.
CreateTicket	פתיחת פנייה לתמיכה.

דוגמאות לפעולות שירות

שכבת השירות היא הגשר בין הלקוחות לבין בסיס הנתונים. הלקוחות קוראים לפעולות שנמצאות ב-Service1, והמחלקה Service1 מבצעת את העבודה בפועל.

שכבת השירות WCF

לקוח MAUI מתאים למובייל. המטרה שלו היא לתת גישה לפעולות חשובות גם בטלפון, כמו צפייה בשיעורים, תשלום וקבלת התראות.

לקוח MAUI

לקוח Web בנוי ב-Razor Pages. הוא מתאים יותר לניהול דרך דפדפן. בדפים האלה יש הפרדה לפי תפקיד, כדי לא להציג למשתמש אפשרויות שלא קשורות אליו.

לקוח Web

לקוח WPF הוא הלקוח המרכזי למחשב. הוא כולל מסכים לתלמיד ולמורה, כמו שיעורים, תשלומים, התראות, קורסים ותמיכה. היתרון שלו הוא שהוא נוח לעבודה במסך גדול.

לקוח WPF

שכבת הלקוח כוללת את WPF, Web ו-MAUI. התפקיד שלה הוא להציג מסכים למשתמש ולקבל ממנו פעולות. הלקוח לא אמור לשמור נתונים בעצמו, אלא לשלוח בקשות לשרת.

שכבת הלקוח

בחלק הזה יש פירוט נוסף על הצד הטכני של הפרויקט, אבל בשפה פשוטה כדי שיהיה קל להבין מה כל שכבה עושה.

חלק 9: פירוט טכני נוסף

פניית תמיכה מאפשרת למשתמש להסביר בעיה. הפנייה נשמרת בבסיס הנתונים, והמנהל יכול לראות אותה, לענות עליה ולסגור אותה. זה נותן למערכת מקום מסודר לטיפול בבעיות במקום הודעות חיצוניות.

תהליך פניות תמיכה

פעולה	מי מקבל התראה	למה זה חשוב
ביטול שיעור	מורה	המורה צריך לדעת שהשיעור השתנה.
הודעה מהמורה	תלמיד	התלמיד מקבל עדכון ישיר מהמורה.
פנייה לתמיכה	מנהל	המנהל יודע שיש בעיה שצריך לטפל בה.

התראה נשמרת בטבלת Notifications עם המשתמש המקבל, סוג המשתמש, תוכן ההודעה והאם היא נקראה. כך כל משתמש רואה רק את ההתראות שלו.

תהליך התראות

ביטול שיעור לא חייב למחוק את השיעור לגמרי. במקרים כאלה עדיף לסמן את השיעור כמבוטל, כדי שתישאר היסטוריה. בנוסף, המערכת יכולה ליצור התראה למורה כדי שהוא ידע שהשיעור השתנה.

תהליך ביטול שיעור

כאשר תלמיד משלם על שיעור, המערכת שומרת רשומת תשלום בטבלת Payments ומעדכנת את השיעור כך שיהיה ברור שהוא שולם. בצורה הזאת אפשר להציג היסטוריית תשלומים ולחשב דוחות.

תהליך תשלום

החלק החשוב כאן הוא שהשיעור לא נשמר רק במסך. הוא נשמר בבסיס הנתונים, ולכן אפשר לראות אותו גם בלקוח אחר שמחובר לאותו שירות.

בתהליך קביעת שיעור התלמיד בוחר תאריך ושעה. המערכת צריכה לוודא שהנתונים הגיוניים, לשמור את השיעור בטבלת Lessons, ולהציג אותו ברשימת השיעורים של התלמיד והמורה.

תהליך קביעת שיעור

שלב	מה קורה
1	המשתמש מזין שם משתמש וסיסמה.
2	הלקוח שולח בקשה לפעולת התחברות ב-Service1.
3	השרת בודק את הנתונים מול הטבלאות Student ו-Teacher.
4	אם המשתמש קיים, המערכת בודקת אם הוא תלמיד, מורה או מנהל.
5	הלקוח מציג את המסך המתאים לתפקיד.

המשתמש מכניס שם משתמש וסיסמה. הלקוח שולח את הפרטים לשרת WCF. השרת בודק את המשתמש מול בסיס הנתונים, ואם הפרטים נכונים הוא מחזיר תשובה ללקוח. לפי התפקיד של המשתמש, הלקוח מעביר אותו למסך מתאים.

תהליך התחברות והרשאות

בחלק הזה אני מפרט יותר את התהליכים המרכזיים במערכת. המטרה היא להראות שהפרויקט לא בנוי רק ממסכים, אלא מתהליך שלם שמתחיל אצל המשתמש ומסתיים בשמירה או שליפה של נתונים.

חלק 11: סיכום ונספחים

בפרויקט הזה למדתי איך לחבר כמה לקוחות שונים לאותו שרת, ואיך לשמור נתונים בצורה מסודרת בבסיס נתונים. החלק שהיה לי הכי מאתגר היה לגרום לכל הלקוחות לעבוד עם אותו שירות בלי לשבור תאימות. אני חושב שהפרויקט מתאים לנושא שבחרתי כי הוא פותר בעיה אמיתית שנתקלתי בה בעצמי. אם הייתי ממשיך את הפרויקט, הייתי משפר עוד את עיצוב המסכים ומוסיף אפשרות לסנכרון עם יומן חיצוני.

מה מצורף בסוף

- קוד הפרויקט המלא נמצא בתיקיות הפרויקט ולא הועתק כולו לתוך הספר.
- הקבצים המרכזיים: Service1.cs, IService1.cs, מחלקות Model, מחלקות ViewDB, מסכי WPF, דפי Web ודפי MAUI.
- מפות הניווט נמצאות גם בתיקיית nav-map.
- ביבליוגרפיה קצרה
 - Microsoft Learn - WCF, WPF, Razor Pages, MAUI.
 - חומרי הלימוד של מגמת הנדסת תוכנה.
 - תיעוד Microsoft Access ו-OleDb.